Integration Reports Guide

Ambitious about Autism — Training Platform

How to pull platform data into **Microsoft Excel**, **Power BI**, and **Microsoft Dynamics 365** using the platform's Integration API.

Table of Contents

---

1. Overview

The Integration API is a read-only HTTPS endpoint that exports platform data in a format designed for Microsoft business intelligence tools. It is not a streaming or real-time API — clients pull on a schedule (typically daily) and the platform returns the current state.

What you can pull

Section	What it contains	How big it gets
`training`	Per-organisation × per-module completion stats. One row per (org, module)	Bounded: roughly `orgs × modules`
`library`	Per-document download counts and metadata	Bounded: ~documents in the library
`surveys`	Every completed survey response, flattened to one row per (response × question)	Largest — grows linearly with responses
`cv`	CV Builder usage — one row per CV. Metadata only (status, template, counts, timestamps). No CV content.	Roughly one per active CV
`careers`	Careers Advisor usage — one row per session. Metadata only. No report content.	Roughly one per session

What the API does well

- **Stable contract.** Every response carries `apiVersion: 'v1'` and an ISO `generatedAt` timestamp.
- **Two formats.** Use `?format=flat` for tabular BI (one row per measurement with a stable `rowId` primary key). Use `?format=nested` for hierarchical JSON.
- **Incremental refresh** via `?since=<ISO datetime>` so daily pulls only ship the new rows.
- **Pagination** on surveys via `?limit=` + `?cursor=` so a survey with thousands of responses doesn't time out.
- **Cacheable.** Every response carries a weak `ETag`. Pass `If-None-Match` on the next poll and the server returns 304 (no body) if nothing has changed.
- **OpenAPI 3.0 schema** at `/api/integrations/reports/schema` — drop the URL into a Power BI or Dynamics Custom Connector "Import from URL" flow.

What the API doesn't do (yet)

- **No webhooks.** Polling only. Daily is a sensible cadence.
- **No write access.** Read-only.
- **No raw user identifiers.** All `userId`-shaped fields are pseudonymised (see section 13).

- **No CV content or careers report content.** Just metadata, status, and counts.

---

2. Before you start

You need:

1. **A user account with Charity Admin role** on the platform (you can manage API keys at `/super-admin/integrations`)
2. **The deployed URL** of the platform. In the examples below we use `https://asd-training-app-v2.vercel.app` — substitute your environment's URL
3. **A tool that can make authenticated HTTPS GET requests.** Anything modern works — Excel 365, Power BI Desktop, Power Automate, Postman, `curl`

The data is pulled with a `Authorization: Bearer <key>` header. Keys do not expire unless an expiry is set when they are created, and they can be revoked at any time.

---

3. Generate an API key

1. Sign in as a Charity Admin
2. Open **Integrations** in the sidebar (or visit `/super-admin/integrations`)
3. Click **New API key**
4. Give the key a descriptive name (e.g. "*Power BI service refresh*" or "*Dynamics Custom Connector*") — this is what you'll see in the audit log
5. Optionally set an expiry date
6. Click **Create**

Important: The full key is shown **once only**. Copy it immediately and store it somewhere secure (a password manager, Azure Key Vault, a Power BI deployment-pipeline secret, etc.). If you lose the key, revoke it and generate a new one.

The key looks like `int_d3a8...` (62 hex characters after the `int_` prefix).

In the keys list you will see:

- A short **prefix** (the first 12 characters) for identification
- The key name

- Created at / expires at / last used at — useful for auditing

---

4. Test the API works

Before configuring Excel or Power BI, confirm the key works using `curl` or any HTTP client.

```
curl -H "Authorization: Bearer int_YOUR_KEY_HERE" \
  "https://asd-training-app-v2.vercel.app/api/integrations/reports?
section=training&format=flat"
```

A successful response looks like:


```
{
  "apiVersion": "v1",
  "generatedAt": "2026-05-16T11:45:00.000Z",
  "section": "training",
  "format": "flat",
  "incrementalSupported": false,
  "since": null,
  "rows": [
    {
      "rowId": "org_abc:mod_xyz",
      "organisationId": "org_abc",
      "organisationName": "Example Trust",
      "organisationSlug": "example-trust",
      "moduleId": "mod_xyz",
      "moduleName": "Introduction to ASD",
      "programName": "ASD Awareness Training",
      "gatsbyBenchmarks": ["3", "4"],
      "totalUsers": 24,
      "completions": 18,
      "completionRate": 75
    }
  ],
  "rowCount": 47
}
```

If you get a `401 Unauthorized`, the key is wrong, revoked, or expired. If you get redirected to `/login` or see HTML, the platform middleware is intercepting the request — make sure you're using the production URL and the Bearer header is set.

You can also pull the OpenAPI schema (no auth needed) to confirm the endpoint is up:

```
curl https://asd-training-app-
v2.vercel.app/api/integrations/reports/schema
```

---

5. Microsoft Excel — Power Query

Excel 365 and Excel 2019+ (Windows) can pull the API directly using Power Query. The result is a refreshable table on a worksheet.

Step-by-step

1. Open Excel and create a new workbook
2. **Data** ribbon → **Get Data** → **From Other Sources** → **Blank Query**
3. In the Power Query editor that opens, click **Advanced Editor**
4. Replace the contents with the M code below, substituting your URL and key:

```
let
    BaseUrl = "https://asd-training-app-
v2.vercel.app/api/integrations/reports",
    ApiKey = "int_YOUR_KEY_HERE",
    Source = Json.Document(
        Web.Contents(
            BaseUrl,
            [
                Query = [section = "training", format = "flat"],
                Headers = [Authorization = "Bearer " & ApiKey]
            ]
        )
    ),
    Rows = Source[rows],
    Table = Table.FromList(Rows, Splitter.SplitByNothing(), null, null,
ExtraValues.Error),
    Expanded = Table.ExpandRecordColumn(
        Table, "Column1",

{"rowId","organisationId","organisationName","moduleId","moduleName","progra
    )
in
    Expanded
```

5. Click **Done**, then **Close & Load**
6. The data appears on a worksheet as a refreshable table. Right-click → **Refresh** to pull fresh data.

Different sections

Change the `section` value in the query to pull different data:

- `"training"` — completion stats per org/module
- `"library"` — document download counts
- `"surveys"` — flattened survey responses
- `"cv"` — CV Builder usage
- `"careers"` — Careers Advisor usage

Storing the key safely

Hard-coding the key in the M expression is fine for a personal workbook but **don't share or email the workbook with the key inside**. For shared workbooks, use Power Query parameters and Excel's data source credentials manager:

1. **Data** → **Get Data** → **Data Source Settings** → **Edit Permissions** on the API URL
2. Set authentication type to **Web API** and paste your key
3. Excel stores the key per-user; recipients of the workbook supply their own key

---

6. Power BI Desktop — basic refresh

Power BI Desktop is the most common tool for serious reporting against this API. The simplest setup uses the **Web** connector with custom headers — no Custom Connector required.

Step-by-step

1. Open **Power BI Desktop**
2. **Home** ribbon → **Get data** → **Web**
3. Switch to **Advanced**
4. **URL parts** — single field: `https://asd-training-app-v2.vercel.app/api/integrations/reports?section=training&format=flat`
5. **HTTP request header parameters**:
   - Name: `Authorization`
   - Value: `Bearer int_YOUR_KEY_HERE`
6. Click **OK**
7. Power BI will preview the JSON; click **Transform Data**
8. In Power Query, drill into `rows` (it's a list of records)
9. Convert the list to a table and expand the record columns
10. **Close & Apply**

You now have a refreshable Power BI table. To add another section, repeat with a different `section=` value.

One query per section

The flat format puts each section into one rectangular table. The cleanest setup is one Power BI query per section:

- `qry_training`
- `qry_library`
- `qry_surveys`
- `qry_cv`
- `qry_careers`

These don't join in Power BI — they are independent fact tables. The shared dimensions (organisations, dates, roles) emerge from the data itself.

Pulling everything in one go

If you want one query that fetches all sections, omit the `section` parameter (or use `section=all`). The response is shaped differently — it has top-level keys `training`, `library`, `surveys`, `cv`, `careers` each containing `rows`. You'll need to expand each in Power Query separately.

For most BI workflows, one-section-per-query is cleaner.

---

7. Power BI — incremental refresh

Power BI's **Incremental Refresh** feature can use the API's `since` parameter to refresh only the new rows on each scheduled run. This matters when survey responses run into the thousands.

What supports incremental refresh

Section	Incremental?	Why
`training`	**No** — aggregates are full-population	Completion rates depend on all-time data
`library`	Yes — filters download events by `createdAt`	
`surveys`	Yes — filters responses by `completedAt`	
`cv`	Yes — filters CVs by `updatedAt`	

`careers`	Yes — filters sessions by `updatedAt`	

Training is small enough to fully refresh each cycle. For the others, use incremental refresh once response volume crosses a few thousand rows.

Setup recipe (surveys section)

1. In Power BI Desktop, create two parameters (**Home** → **Transform data** → **Manage Parameters**):
   - `RangeStart` — type `Date/Time`, default e.g. 1 Jan 2020
   - `RangeEnd` — type `Date/Time`, default today
2. Edit your `qry_surveys` M expression to use these parameters:

```
let
    BaseUrl = "https://asd-training-app-
v2.vercel.app/api/integrations/reports",
    ApiKey = "int_YOUR_KEY_HERE",
    SinceText = DateTime.ToText(RangeStart, "yyyy-MM-ddTHH:mm:ssZ"),
    Source = Json.Document(
        Web.Contents(
            BaseUrl,
            [
                Query = [section = "surveys", format = "flat", since =
SinceText, #"limit" = "5000"],
                Headers = [Authorization = "Bearer " & ApiKey]
            ]
        )
    ),
    Rows = Source[rows],
    Table = Table.FromList(Rows, Splitter.SplitByNothing(), null, null,
ExtraValues.Error),
    Expanded = Table.ExpandRecordColumn(
        Table, "Column1",

{"rowId","surveyId","surveyTitle","respondentId","role","organisation","comp
    ),
    Typed = Table.TransformColumnTypes(Expanded, {{"completedAt", type
datetimezone}}),
    Filtered = Table.SelectRows(Typed, each [completedAt] >= RangeStart
and [completedAt] < RangeEnd)
in
    Filtered
```

3. **Close & Apply**
4. Right-click the table in the Fields pane → **Incremental refresh**
5. Configure:
    - **Store rows from the past** — e.g. 5 years
    - **Refresh rows from the past** — e.g. 7 days (Power BI re-fetches the trailing window each refresh)
6. Publish to the Power BI Service

Handling pagination

The API caps a single survey-section response at `limit=5000` rows. If a single refresh window genuinely returns more than 5000 rows (rare for a daily refresh), you'll see a `nextCursor` field in the response. You can either:

- **Easier:** narrow the refresh window so each batch stays under the cap
- **Robust:** wrap the M expression in a recursive loop that follows `nextCursor` until null (advanced; ask your BI lead to implement a paged-fetch helper function)

---

8. Power BI Service — scheduled refresh

For Power BI reports published to the Service, refresh runs in the cloud — not on your desktop.

Authentication

Power BI Service supports the same Web connector authentication as Desktop:

1. Publish your report from Desktop to a workspace
2. Open the **dataset** in the Service → **Settings** → **Data source credentials**
3. Edit credentials for the API URL → set authentication to **Anonymous** (the Bearer token is in the request header, not a separate credential)
4. The header from your M expression travels with the request

Power BI Service does **not** prompt for the Bearer key separately — it relies on the header you specified in M. This is why you can't hard-code the key in plain text and share the dataset; the key lives in the dataset definition.

Best practice: for production, store the key in **Azure Key Vault** and load it into M via the Power BI Pro / Premium **dataflow parameters** feature. This keeps the key out of the report file.

Schedule

In dataset **Settings** → **Scheduled refresh**, set the cadence. Most teams use:

- **Daily** at a quiet hour (e.g. 4 am UTC) — generous window for incremental refresh
- **Twice daily** if survey response timeliness matters

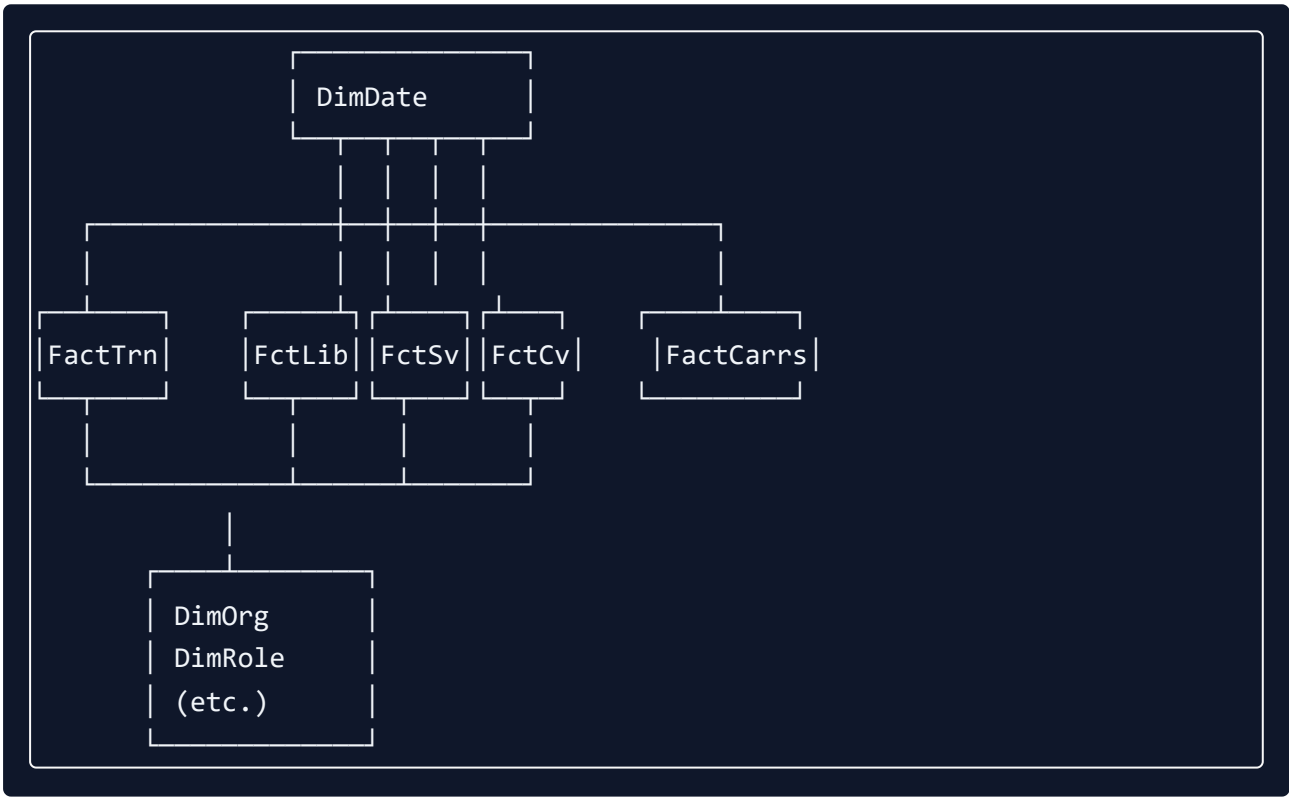
There is no platform-side gateway requirement — the API is a public HTTPS endpoint accepting Bearer auth, so no on-premises gateway is needed.

---

9. Building Power BI reports — modelling tips

Once the data is in Power BI, treat each section as an independent fact table. Use the platform's natural shape:

Recommended star schema



Dimension table tips

- **DimOrg** — derive from any fact table's distinct `(organisationId, organisationName)` pairs. Or pull all five sections into a staging query, union the org columns, and dedupe.
- **DimRole** — small fixed enum: `SUPER_ADMIN`, `CHARITY_EMPLOYEE`, `ORG_ADMIN`, `CAREGIVER`, `CAREER_DEV_OFFICER`, `STUDENT`, `INTERN`, `EMPLOYEE`. The training section excludes the two admin roles from `totalUsers` — be aware when joining.

- **DimDate** — standard date dimension keyed on `completedAt` (surveys), `updatedAt` (CV/careers), `createdAt` (library events).

Common measures (DAX)

```
Training Completion Rate =
    DIVIDE(SUM(FactTraining[completions]), SUM(FactTraining[totalUsers]))

Active Surveys (last 30 days) =
    CALCULATE(
        DISTINCTCOUNT(FactSurvey[surveyId]),
        FactSurvey[completedAt] >= TODAY() - 30
    )

CVs Completed This Month =
    CALCULATE(
        COUNTROWS(FactCv),
        FactCv[status] = "COMPLETE",
        DATESINPERIOD(DimDate[Date], TODAY(), -30, DAY)
    )

Unique CV Authors =
    DISTINCTCOUNT(FactCv[userPseudonym])

Unique Survey Respondents (this survey) =
    DISTINCTCOUNT(FactSurvey[respondentId])
```

Important on pseudonyms: because survey pseudonyms are namespaced per-survey, the same person responding to two surveys appears as two different pseudonyms. You **cannot** count distinct respondents across surveys in this dataset. That's deliberate — it prevents cross-survey re-identification. See section 13.

Survey response shape

The survey flat format puts **one row per (response × question)**, which is Power BI's preferred long format. This means:

- A respondent's answers are spread across multiple rows
- To get a single respondent's full answers, filter on their `respondentId`

- To pivot for export, use Power BI's matrix visual or DAX `PIVOT` patterns

---

10. Microsoft Dynamics 365 — Custom Connector

For Dynamics 365 (Customer Engagement, Customer Insights, Sales) and Power Platform broadly, the cleanest integration is a **Custom Connector** that wraps the API. Once built, the connector appears as an action in Power Automate, Power Apps, and Power BI.

Build the connector

1. Sign in to **Power Apps** at `https://make.powerapps.com`
2. **Data** → **Custom connectors** → **New custom connector** → **Create from OpenAPI URL**
3. **OpenAPI URL**: `https://asd-training-app-v2.vercel.app/api/integrations/reports/schema`
4. Give the connector a name (e.g. "*AAA Reporting*")
5. Power Platform parses the schema and generates the connector
6. On the **Security** step:
   - Authentication type: **API Key**
   - Parameter label: `API Key`
   - Parameter name: `Authorization`
   - Parameter location: `Header`
   - When a flow uses the connector, supply the value as `Bearer int_YOUR_KEY_HERE`
7. **Create connector**
8. **Test** tab → **New connection** → paste the Bearer-prefixed key → run a sample call

Using the connector

Once published, the connector appears in Power Automate's connector picker. A common pattern:

- **Trigger**: Recurrence (daily)
- **Action 1**: AAA Reporting — Get reports (`section=surveys` , `format=flat` , `since=` formula `formatDateTime(addDays(utcNow(), -1), 'yyyy-MM-ddTHH:mm:ssZ')`)

- **Action 2**: Parse JSON
- **Action 3**: For each row → upsert into Dataverse or Dynamics entity

Map to Dataverse entities

Because every flat row has a stable `rowId`, you can use **upsert** patterns in Dataverse:

- Set the row's external key column to `rowId`
- Use Power Automate's **Dataverse — Add or update a row** action
- Map the rest of the columns to Dataverse fields

This makes daily syncs idempotent — re-running yesterday's sync doesn't create duplicates.

---

11. Power Automate flows

Some recipes that don't need Power BI or Dynamics:

Daily survey response digest to email

1. **Trigger**: Recurrence (daily, 8 am UK time)
2. **HTTP** action (or your Custom Connector): GET `/api/integrations/reports?section=surveys&format=flat&since=` *(24h ago)*
3. **Parse JSON** with the schema from `/api/integrations/reports/schema`
4. **Condition**: if `rowCount > 0`, send email with the row count and link to the platform's results page

Notify Slack when a new high-priority survey closes

1. **Trigger**: Recurrence (hourly)
2. **HTTP** GET `/api/integrations/reports?section=surveys` (no `since` — full state)
3. **Parse JSON**
4. Filter for surveys with `status: CLOSED` and `closesAt` in the last hour
5. **Post message in a channel** action (Slack connector)

Sync CV completion stats to a SharePoint list

1. **Trigger**: Recurrence (daily)
2. **HTTP** GET `/api/integrations/reports?section=cv&format=flat&since=` *(yesterday)*
3. **For each** row → **SharePoint — Create item** or **Update item** keyed on `rowId`

---

12. Data reference

What each section returns. The full machine-readable contract is at `/api/integrations/reports/schema`.

`training`

One row per `(organisation, module)`.

Column	Type	Notes
`rowId`	string	`<organisationId>:<moduleId>`
`organisationId`	string	
`organisationName`	string	
`organisationSlug`	string	URL-safe org identifier
`moduleId`	string	
`moduleName`	string	
`programName`	string	e.g. "ASD Awareness Training"
`gatsbyBenchmarks`	string[]	UK Gatsby Benchmarks 1–8 the module maps to
`totalUsers`	integer	**Excludes** SUPER_ADMIN and ORG_ADMIN roles
`completions`	integer	Users who have completed at least one lesson in the module
`completionRate`	integer	0–100

Cohort orgs are excluded so the figures match the in-app super-admin reports.

`incrementalSupported: false` — full state every call.

`library`

One row per `(collection, document)`.

Column	Type	Notes
`rowId`	string	Document id
`collectionId`	string	
`collectionTitle`	string	
`collectionActive`	boolean	Whether the collection is published
`documentId`	string	
`documentTitle`	string	
`fileName`	string	Original upload filename
`downloads`	integer	Lifetime; respects `?since=` for filtering

`incrementalSupported: true` — pass `?since=` to limit the download counts to events on or after that timestamp.

`surveys`

One row per `(response × question)` — Power BI's preferred long format.

Column	Type	Notes
`rowId`	string	`<responseId>:<questionId>`
`surveyId`	string	
`surveyTitle`	string	
`surveyStatus`	string	`DRAFT` | `PUBLISHED` | `CLOSED`
`respondentId`	string	**Pseudonymised.** Stable per `(user, survey)` — do not assume the same `respondentId` across surveys means the same person
`role`	string	Real role at time of response

`organisation`	string	Org name at time of response
`completedAt`	datetime (ISO)	
`questionId`	string	
`question`	string	Question text
`questionType`	string	`MULTIPLE_CHOICE` | `YES_NO` | `FREE_TEXT` | `RATING_SCALE` | `MULTI_SELECT`
`answer`	string	The actual answer; for multi-select this is comma-separated

`incrementalSupported: true` via `?since=` on `completedAt`. Paginated via `?limit=` + `?cursor=`.

`cv`

One row per CV. **No CV content** is exposed — just metadata.

Column	Type	Notes
`rowId`	string	CV id
`cvId`	string	
`userPseudonym`	string	**Pseudonymised.** Stable per user across all their CVs. Not joinable to careers or survey pseudonyms
`role`	string	
`organisationId`	string | null	
`organisationName`	string	
`status`	string	`DRAFT` | `COMPLETE`
`template`	string	`ACCESSIBLE` | `MODERN` | `CLASSIC`
`currentStep`	integer	0–8 in the wizard

`workExperienceCount`	integer	
`educationCount`	integer	
`skillsCount`	integer	
`createdAt`	datetime (ISO)	
`updatedAt`	datetime (ISO)	

`incrementalSupported: true` via `?since=` on `updatedAt`.

`careers`

One row per Careers Advisor session. **No report content** is exposed — just metadata.

Column	Type	Notes
`rowId`	string	Session id
`sessionId`	string	
`userPseudonym`	string	**Pseudonymised.** Stable per user across all their sessions. Not joinable to CV or survey pseudonyms
`role`	string	
`organisationId`	string | null	
`organisationName`	string	
`status`	string	`IN_PROGRESS` | `COMPLETE`
`currentStep`	integer	0–11 in the wizard
`hasReport`	boolean	True once the AI report has been generated
`createdAt`	datetime (ISO)	
`updatedAt`	datetime (ISO)	

`incrementalSupported: true` via `?since=` on `updatedAt`.

---

13. Privacy notes — what is and isn't exposed

What's pseudonymised

All user-identifying fields are replaced with a stable, non-reversible pseudonym:

- **Survey responses** — `respondentId` is HMAC-SHA-256 of `(surveyId, userId)` truncated to 16 hex characters
- **CV Builder rows** — `userPseudonym` is HMAC-SHA-256 of `('cv', userId)`
- **Careers Advisor rows** — `userPseudonym` is HMAC-SHA-256 of `('careers', userId)`

The HMAC secret is the platform's `NEXTAUTH_SECRET` and is never transmitted. Pseudonyms cannot be reversed from a leaked report.

Why pseudonyms differ across sections

Because the namespace differs, the same user has a different pseudonym in surveys vs CVs vs careers. This is **intentional** — it prevents someone with two leaked reports from cross-referencing them to re-identify a person.

Consequence: you cannot calculate "users who have both completed a CV and a survey" from this dataset. If you need that figure, request it through the in-app Reports page instead, where the underlying user identity is available to the Charity Admin viewing the data.

What's not exposed

- Email addresses
- Names
- CV personal-detail fields (name, address, phone, work history descriptions, personal statement, etc.)
- Careers Advisor questionnaire answers
- Careers Advisor AI-generated report content
- Survey free-text answers from non-completed responses (only `completedAt: not null` rows are returned)

- User passwords or any auth tokens

What is exposed in plain text

- Organisation names and slugs
- Role names (e.g. `STUDENT`)
- Module / lesson / program names
- Library document titles and filenames
- Survey titles, question text, and answer values
- Counts and timestamps everywhere

If your data-protection officer wants the formal record, see `docs/compliance/ROPA.md` and `docs/compliance/DPIA.md` in the repository.

---

14. Troubleshooting

401 Unauthorized

- The Bearer token is missing, wrong, expired, or revoked
- Check the key prefix in `/super-admin/integrations` matches the first 12 characters of the key you're using
- Make sure the header is exactly `Authorization: Bearer int_...` (note the space after `Bearer`)

429 Too Many Requests

- You've exceeded 60 requests per minute on this key
- The response includes a `Retry-After` header (seconds to wait)
- If you're hitting this regularly, batch your refreshes or split into multiple keys (one per consumer)

304 Not Modified (and no data)

- This is **not an error** — the server is telling you nothing has changed since the ETag you supplied
- If your client requested data with `If-None-Match: <etag>` and the dataset hasn't changed, you get 304 with an empty body
- Use the data you already have

400 Bad Request — invalid section

- The `?section=` value is not one of `training`, `library`, `surveys`, `cv`, `careers`, `all`
- Check spelling and casing (lower-case)

Power BI: "Web.Contents failed to get contents from..."

- Usually means the URL is wrong or the Bearer header isn't being sent
- In Power BI, use the **Advanced** Web connector mode (not the basic single-line URL) so you can set headers
- For the Service, the credentials must be set to **Anonymous** (the header carries the auth)

Power BI: "Refresh failed because the data source is in a different region"

- The platform is hosted on Vercel. If your tenant is in a non-default region, you may need a gateway — but most setups work without one
- Try setting the data source privacy level to **Public** in the Service settings

Excel: data loads once but Refresh fails

- Your stored Bearer key has expired or been revoked — visit **Data → Data Source Settings** and edit the credentials

Dynamics Custom Connector test returns 401

- The Security tab parameter must be configured as a **header** named `Authorization` with the value `Bearer int_YOUR_KEY` (including the `Bearer` prefix)

- A common mistake is putting just the key with no `Bearer` prefix

"I'm seeing different numbers than the in-app reports"

- **Training completion** — the in-app reports exclude cohort orgs; the API now does the same. If you see a mismatch, check the deployment is on the latest version (see CLAUDE.md changelog).
- **Survey respondent counts** — the API only returns responses where `completedAt` is set. The in-app reports may also count in-progress responses.
- **User counts** — the API's `training.totalUsers` excludes `SUPER_ADMIN` and `ORG_ADMIN`. Compare like-for-like.

---

15. FAQ

How often should I refresh?

- **Daily** is the typical cadence for charity-wide reporting
- **Hourly** is fine if your survey response volumes are high and timeliness matters
- **More frequently than that** burns through the 60 req/min limit quickly across multiple consumers — coordinate keys and schedules

Can I push data INTO the platform via this API?

No — this API is read-only by design. To create or modify data, sign in as a Charity Admin and use the platform UI, or build a separate write integration (not part of this guide).

What happens if my key is leaked?

1. Sign in as Charity Admin → **Integrations** → revoke the key (one click)
2. Generate a new key
3. Update Power BI / Excel / Dynamics with the new key
4. Review the audit log (`lastUsedAt` per key) for unexpected usage

Leaked keys do **not** expose passwords or auth tokens — only the pseudonymised reporting data this guide describes.

Can I have one key per consumer (Power BI, Dynamics, etc.)?

Yes — and you should. Per-consumer keys mean you can revoke and rotate one without affecting the others, and the audit log shows which consumer was actually hitting the API.

Why is the survey data so big?

Because it's in long format — one row per (response × question). A 20-question survey with 500 respondents is 10,000 rows. Power BI and Dataverse handle this fine; spreadsheets struggle around 1 million rows.

If you need an aggregate view for spreadsheet consumption, build a Power BI dataset and export the aggregated tables — not the raw rows.

What if a new field appears in a response I'm parsing?

The API guarantees backwards-compatible additions within `apiVersion: 'v1'` — new fields may appear, but existing fields will not change shape or be removed without a version bump. Your Power Query / Custom Connector code should ignore unknown fields by default (which is the default behaviour for Power Query record expansion and Dataverse mapping).

If we ever need to break the contract, you'll see `apiVersion: 'v2'` and a deprecation window will be communicated to integration partners.

Who do I contact?

For platform issues, contact the platform admin team at the charity. For BI-side issues (Power BI authoring, Custom Connector debugging), your internal IT or BI partner is best placed to help.

Last updated: May 2026 — API v1